

Módulo 3: Algoritmos de Machine Learning – Regresión

Material de Evaluación y Práctica

Especialista en Inteligencia Artificial

2026

Resumen

Este documento contiene 10 ejercicios prácticos diseñados para evaluar y fortalecer las capacidades cognitivas en el área de algoritmos de regresión. Los ejercicios siguen una curva de dificultad progresiva y se basan en casos de uso reales.

Índice

1. Fundamentos Teóricos y Criterios de Selección	3
1.1. Diferenciación de Algoritmos de Regresión	3
1.2. Criterios de Elección del Modelo	3
2. Métricas de Evaluación en Regresión	4
2.1. Métricas de Error (Magnitud del Fallo)	4
2.2. Caso de Estudio: Entregas de E-commerce	4
2.2.1. Modelo A (Errores pequeños y constantes)	5
2.2.2. Modelo B (Predicciones perfectas con 1 fallo catastrófico)	5
2.2.3. Conclusión Analítica para el Alumno:	5
2.3. Métricas de Ajuste (Bondad de Ajuste)	6
2.4. Métricas Específicas por Algoritmo	6
2.5. Ejemplo Práctico: El Coste de la Energía	6
3. Ejercicios Prácticos y Soluciones	7
3.1. Ejercicio 1 - Regresión Lineal Simple (Scikit-Learn)	7
3.1.1. Enunciado del Reto	7
3.1.2. Solución en Python	7
3.1.3. Resultados de la Ejecución	7
3.1.4. Comentario Analítico	7
3.2. Ejercicio 2 - Regresión Lineal con Statsmodels	8
3.2.1. Enunciado del Reto	8
3.2.2. Solución en Python	8
3.2.3. Resultados de la Ejecución	8
3.2.4. Comentario Analítico	9
3.3. Ejercicio 3 - Ajuste de Mínimos Cuadrados (Fundamentos)	9
3.3.1. Enunciado del Reto	9
3.3.2. Solución en Python	9
3.3.3. Resultados de la Ejecución	10
3.3.4. Comentario Analítico	10
3.4. Ejercicio 4 - Regresión Lineal Múltiple	10
3.4.1. Enunciado del Reto	10
3.4.2. Solución en Python	11

3.4.3.	Resultados de la Ejecución	11
3.4.4.	Comentario Analítico	11
3.5.	Ejercicio 5 - Tratamiento de Variables Categóricas en Regresión	11
3.5.1.	Enunciado del Reto	12
3.5.2.	Solución en Python	12
3.5.3.	Resultados de la Ejecución	12
3.5.4.	Comentario Analítico	12
3.6.	Ejercicio 6 - Regresión Polinomial	13
3.6.1.	Enunciado del Reto	13
3.6.2.	Solución en Python	13
3.6.3.	Resultados de la Ejecución	13
3.6.4.	Comentario Analítico	13
3.7.	Ejercicio 7 - Análisis de Complejidad y Sobreajuste (Overfitting)	14
3.7.1.	Enunciado del Reto	14
3.7.2.	Solución en Python	14
3.7.3.	Resultados de la Ejecución	14
3.7.4.	Comentario Analítico	14
3.8.	Ejercicio 8 - Regresión Logística (Introducción)	15
3.8.1.	Enunciado del Reto	15
3.8.2.	Solución en Python	15
3.8.3.	Resultados de la Ejecución	15
3.8.4.	Comentario Analítico	15
3.9.	Ejercicio 9 - Evaluación de Regresión Logística	16
3.9.1.	Enunciado del Reto	16
3.9.2.	Solución en Python	16
3.9.3.	Resultados de la Ejecución	16
3.9.4.	Comentario Analítico	17
3.10.	Ejercicio 10 - Creación de un Pipeline de Regresión Completo	17
3.10.1.	Enunciado del Reto	17
3.10.2.	Solución en Python	17
3.10.3.	Resultados de la Ejecución	18
3.10.4.	Comentario Analítico	18
4.	Actividades de Consolidación	19

1. Fundamentos Teóricos y Criterios de Selección

Para un Especialista en IA, no basta con aplicar algoritmos; es fundamental entender qué modelo es el más adecuado para cada problema. A continuación, se presenta una guía de diferenciación y selección.

1.1. Diferenciación de Algoritmos de Regresión

Cuadro 1: Comparativa de Algoritmos de Regresión e Inferencia

Algoritmo	Descripción	Casos de Uso
Regresión Lineal Simple	Modela la relación entre una variable independiente (X) y una dependiente (y) mediante una línea recta.	Relaciones directas (ej. inversión vs. ventas).
Regresión Lineal Múltiple	Extiende la anterior al usar múltiples variables predictoras ($X_1, X_2, \dots, X_n$).	Escenarios complejos (ej. precio vivienda basado en m^2 , zona, edad).
Regresión Polinomial	Transforma las variables para modelar relaciones curvas mediante potencias de X .	Fenómenos con aceleración o saturación (ej. crecimiento bacteriano).
Regresión Logística	A pesar de su nombre, se usa para clasificación . Predice la probabilidad de pertenecer a una clase binaria usando una función sigmoide.	Decisiones sí/no (ej. conceder crédito, detectar spam).
Statsmodels vs Scikit-Learn	Statsmodels se enfoca en la inferencia estadística (p-valores, intervalos de confianza), mientras que Scikit-Learn se orienta a la capacidad predictiva y el machine learning puro.	Diagnóstico de variables vs. Producción de modelos.

1.2. Criterios de Elección del Modelo

La elección del algoritmo debe basarse en los siguientes factores:

1. Naturaleza de la Variable Objetivo (y):

- Si y es continua (un número real): Usa **Regresión Lineal o Polinomial**.
- Si y es categórica (0/1, Sí/No): Usa **Regresión Logística**.

2. Linealidad de los Datos:

- Realiza un análisis visual previo (Scatter Plot). Si los puntos siguen una línea, la regresión lineal es óptima. Si presentan una curvatura, opta por la polinomial.

3. Complejidad y Overfitting:

- Modelos con grados polinomiales muy altos tienden a memorizar el ruido (**Overfitting**). Busca siempre el modelo más simple que explique bien los datos (Principio de Parsimonia).

4. Explicabilidad vs. Precisión:

- Si necesitas justificar legal o éticamente una decisión (ej. por qué se denegó un crédito), usa modelos lineales con **Statsmodels** para analizar la importancia de cada variable.
- Si solo importa el acierto en la predicción final, usa **Scikit-Learn** y técnicas de validación cruzada.

5. Calidad de los Datos:

- La regresión es sensible a **Outliers** (valores atípicos) y a la **Multicolinealidad** (variables independientes muy correlacionadas entre sí). Siempre preprocesa y limpia tus datos antes de entrenar.

2. Métricas de Evaluación en Regresión

A diferencia de la clasificación, en regresión no medimos “aciertos”, sino la **distancia** o el “error” entre la predicción y el valor real. Entender estas métricas es vital para validar si el modelo es útil para el negocio.

2.1. Métricas de Error (Magnitud del Fallo)

- **MAE (Error Absoluto Medio):** Es el promedio de los errores absolutos. Es la métrica más intuitiva ya que mantiene las mismas unidades que la variable objetivo (y).

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- *Interpretación:* “En promedio, nuestras predicciones se desvían X unidades del valor real”.
- **MSE (Error Cuadrático Medio):** Eleva los errores al cuadrado antes de promediarlos. Al elevar al cuadrado, **penaliza mucho más los errores grandes** que los pequeños (penalización cuadrática).

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- *Interpretación:* Útil cuando queremos evitar a toda costa cometer errores de gran magnitud. Al estar en unidades al cuadrado, no es directamente interpretable en la escala original.
- **RMSE (Raíz del Error Cuadrático Medio):** Es la raíz cuadrada del MSE. Permite interpretar el error penalizado (MSE) en las unidades originales de los datos.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

- *Interpretación:* “Desviación promedio penalizada respecto a la realidad”.

2.2. Caso de Estudio: Entregas de E-commerce

Para entender la diferencia práctica entre MAE y RMSE, imaginemos que entrenamos dos modelos para predecir el tiempo de entrega de paquetes (en horas) para 4 envíos.

- Tiempos reales (y): [24, 48, 72, 96] horas.

2.2.1. Modelo A (Errores pequeños y constantes)

Este modelo siempre se equivoca por exactamente 2 horas en cada entrega (retraso leve constante).

- Predicciones ($\hat{y}_A$): [26, 50, 74, 98] horas.
- Errores absolutos ($|y - \hat{y}|$): [2, 2, 2, 2] $\rightarrow$ Suma = 8.
- Errores al cuadrado ($(y - \hat{y})^2$): [4, 4, 4, 4] $\rightarrow$ Suma = 16.
- MAE: $\frac{8}{4} = 2,0$ horas.
- RMSE: $\sqrt{\frac{16}{4}} = \sqrt{4,0} = 2,0$ horas.

2.2.2. Modelo B (Predicciones perfectas con 1 fallo catastrófico)

Este modelo acierta perfectamente 3 entregas, pero en la última comete un error grave de 8 horas (retraso severo).

- Predicciones ($\hat{y}_B$): [24, 48, 72, 104] horas.
- Errores absolutos ($|y - \hat{y}|$): [0, 0, 0, 8] $\rightarrow$ Suma = 8.
- Errores al cuadrado ($(y - \hat{y})^2$): [0, 0, 0, 64] $\rightarrow$ Suma = 64.
- MAE: $\frac{8}{4} = 2,0$ horas.
- RMSE: $\sqrt{\frac{64}{4}} = \sqrt{16,0} = 4,0$ horas.

2.2.3. Conclusión Analítica para el Alumno:

1. **¿Por qué el MAE es idéntico?** Ambos modelos tienen un MAE de 2,0 horas. Para el MAE, cometer cuatro pequeños fallos de 2 horas es equivalente a cometer un único fallo grande de 8 horas. El MAE es lineal y no discrimina la gravedad o la dispersión de los errores individuales.
2. **¿Por qué el RMSE del Modelo B se duplica?** El RMSE del Modelo B sube a 4,0 horas (el doble que el Modelo A). Al elevar los errores al cuadrado antes de promediar, el error de 8 horas se convierte en 64, castigando severamente al Modelo B.
3. **¿Qué modelo elegir en el negocio?**
 - Si el e-commerce prefiere pequeños retrasos tolerables pero quiere evitar a toda costa un retraso catastrófico que genere reclamaciones legales o pérdida de clientes VIP, debe elegir el Modelo A (menor RMSE).
 - Si al negocio le da igual un fallo grave aislado con tal de que la inmensa mayoría de las entregas sean perfectas y a tiempo, el Modelo B es aceptable.
 - **Regla de oro:** Cuanto mayor sea la brecha entre el RMSE y el MAE en un modelo, mayor presencia de valores atípicos (outliers) existe en sus predicciones.

2.3. Métricas de Ajuste (Bondad de Ajuste)

- **R-cuadrado (R^2):** Conocido como coeficiente de determinación, indica qué porcentaje de la variabilidad de los datos es explicada por el modelo. Va de 0 a 1.
 - *Interpretación:* Un R^2 de 0.90 indica que el modelo explica el 90 % de los cambios observados en los datos.
- **R-cuadrado Ajustado:** Similar al anterior, pero penaliza la inclusión de variables que no aportan valor real. Es la métrica a seguir en la **Regresión Múltiple**.

2.4. Métricas Específicas por Algoritmo

Cuadro 2: Métricas Clave por Algoritmo de Regresión

Algoritmo	Métrica Clave	Cómo interpretarla
Regresión Lineal	MAE / MSE	Buscamos los valores más bajos posibles para asegurar cercanía a los datos reales.
Statsmodels (Inferencia)	p-valor (p-value)	Si $p < 0,05$, aceptamos que la variable independiente tiene un impacto real en la dependiente.
Regresión Polinomial	Curva de Aprendizaje	Comparamos el error en entrenamiento vs. prueba para detectar si el grado del polinomio es excesivo.
Regresión Logística	Log-Loss / Probabilidad	Mide la incertidumbre de la clasificación. A menor Log-Loss, más seguro y acertado es el modelo.

2.5. Ejemplo Práctico: El Coste de la Energía

Imagina un modelo que predice el consumo eléctrico de una fábrica:

- **Escenario 1:** El modelo tiene un MAE bajo. La fábrica puede planificar su presupuesto mensual con una desviación pequeña y conocida.
- **Escenario 2:** El modelo tiene un R^2 alto pero un RMSE muy elevado. Esto indica que el modelo sigue bien la tendencia general, pero en momentos puntuales (ej. picos de producción) comete errores muy grandes que podrían causar apagones o sobrecostes.
- **Conclusión:** Para la estabilidad de la red eléctrica, es preferible optimizar el **RMSE** para minimizar los errores críticos, incluso si el error promedio (MAE) sube ligeramente.

3. Ejercicios Prácticos y Soluciones

3.1. Ejercicio 1 - Regresión Lineal Simple (Scikit-Learn)

Objetivo: Aplicar un modelo de regresión lineal simple para identificar la relación entre una variable independiente y una dependiente en un contexto de marketing.

3.1.1. Enunciado del Reto

Una empresa de e-commerce quiere entender cómo su inversión mensual en publicidad en redes sociales (en miles de dólares) afecta sus ventas totales. Utiliza los datos generados sintéticamente para entrenar un modelo que prediga las ventas basadas en la inversión publicitaria.

3.1.2. Solución en Python

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from sklearn.linear_model import LinearRegression
5
6 # 1. Generación de datos sintéticos
7 np.random.seed(42)
8 inversion = np.random.normal(50, 15, 100).reshape(-1, 1)
9 ventas = 5 + 2.5 * inversion + np.random.normal(0, 10, (100, 1))
10
11 # 2. Creación e entrenamiento del modelo
12 modelo = LinearRegression()
13 modelo.fit(inversion, ventas)
14
15 # 3. Predicción y visualización
16 ventas_pred = modelo.predict(inversion)
17
18 plt.scatter(inversion, ventas, color='blue', label='Datos reales')
19 plt.plot(inversion, ventas_pred, color='red', linewidth=2, label='Línea
    de regresión')
20 plt.title('Impacto de la Publicidad en las Ventas')
21 plt.xlabel('Inversión (Miles $)')
22 plt.ylabel('Ventas (Unidades)')
23 plt.legend()
24 plt.show()
25
26 print(f"Coeficiente (Pendiente): {modelo.coef_[0][0]:.2f}")
27 print(f"Intercepto: {modelo.intercept_[0]:.2f}")
```

3.1.3. Resultados de la Ejecución

```
Coeficiente (Pendiente): 2.40
Intercepto: 9.85
```

3.1.4. Comentario Analítico

- **Interpretación Matemática:** La pendiente estimada de 2,40 indica que por cada unidad adicional en la variable independiente (cada \$1,000 adicionales de inversión publicitaria), las ventas estimadas se incrementan en 2,40 unidades (2,400 productos en promedio). El

intercepto de 9,85 establece la base estimada de ventas (9,850 unidades) si no hubiera inversión publicitaria en redes sociales.

- **Interpretación Práctica:** Esto ayuda a la empresa a calcular su ROI publicitario. La relación positiva y fuerte demuestra la efectividad de las redes sociales para atraer ventas orgánicas basales altas (casi 10,000 unidades) y un crecimiento lineal muy predecible al inyectar presupuesto.

Justificación: El alumno demuestra su capacidad para instanciar, ajustar y visualizar un modelo básico de Scikit-learn, traduciendo una relación teórica en una representación matemática funcional.

3.2. Ejercicio 2 - Regresión Lineal con Statsmodels

Objetivo: Analizar la significancia estadística de los coeficientes de un modelo para validar hipótesis de negocio.

3.2.1. Enunciado del Reto

El departamento de RRHH sospecha que los años de experiencia son el factor determinante en la productividad de los empleados. Utiliza **Statsmodels** para realizar una regresión y analiza el valor p (p-value) para confirmar si esta relación es estadísticamente significativa.

3.2.2. Solución en Python

```
1 import statsmodels.api as sm
2 import numpy as np
3
4 # 1. Datos sintéticos: Años de experiencia vs Score de productividad
5 np.random.seed(42)
6 experiencia = np.random.randint(1, 20, 50)
7 productividad = 20 + 3.5 * experiencia + np.random.normal(0, 5, 50)
8
9 # 2. Preparación de los datos para Statsmodels (añadir constante para el
   intercepto)
10 X = sm.add_constant(experiencia)
11 y = productividad
12
13 # 3. Ajuste del modelo por Mínimos Cuadrados Ordinarios (OLS)
14 modelo_stats = sm.OLS(y, X).fit()
15
16 # 4. Mostrar el resumen estadístico
17 print(modelo_stats.summary())
```

3.2.3. Resultados de la Ejecución

OLS Regression Results		
=====		
Dep. Variable:	y	R-squared:
0.930		
Model:	OLS	Adj. R-squared:
0.929		
Method:	Least Squares	F-statistic:
638.5		

Date:	Wed, 20 May 2026	Prob (F-statistic):			
	2.21e-29				
Time:	10:43:14	Log-Likelihood:			
	-154.79				
No. Observations:	50	AIC:			
	313.6				
No. Residuals:	48	BIC:			
	317.4				
Df Model:	1				
Covariance Type:	nonrobust				
=====					
	coef	std err	t	P> t	[0.025
0.975]					

const	20.2962	1.632	12.435	0.000	17.014
	23.578				
x1	3.5488	0.140	25.270	0.000	3.266
	3.831				
=====					

3.2.4. Comentario Analítico

- **Interpretación Matemática:** El coeficiente de determinación $R^2 = 0,930$ indica que los años de experiencia explican el 93 % de la varianza en la productividad de los empleados, lo cual demuestra un ajuste excepcionalmente alto y robusto. La pendiente para **x1** es de 3,55, y el valor p asociado ($P > |t|$) es de 0,000.
- **Interpretación Práctica:** Al ser el valor p menor a 0,05 (e incluso a 0,01), podemos rechazar la hipótesis nula con una confianza del 99 % y confirmar que los años de experiencia influyen significativamente en la productividad. Por cada año de experiencia extra, la productividad promedio sube 3,55 puntos. La productividad base estimada para empleados sin experiencia es de 20,30 puntos. Esto respalda estadísticamente el desarrollo de planes de carrera en la organización.

Justificación: Esta tarea obliga al alumno a ir más allá de la predicción, interpretando métricas estadísticas críticas como el R-cuadrado y el p-valor para sustentar decisiones corporativas.

3.3. Ejercicio 3 - Ajuste de Mínimos Cuadrados (Fundamentos)

Objetivo: Aplicar la fundamentación matemática de la regresión mediante el cálculo manual de los parámetros en un escenario de consumo energético.

3.3.1. Enunciado del Reto

Imagina que no tienes acceso a librerías de alto nivel. Calcula manualmente la pendiente (m) y el intercepto (b) de la recta de regresión $y = mx + b$ para predecir el consumo eléctrico de un edificio basado en la temperatura exterior, utilizando las fórmulas de mínimos cuadrados.

3.3.2. Solución en Python

```

1 import numpy as np
2
3 # Datos: Temperatura (X) vs Consumo Kw/h (y)
4 X = np.array([15, 20, 25, 30, 35])
5 y = np.array([120, 150, 190, 240, 300])
6
7 # Cálculo de medias
8 x_media = np.mean(X)
9 y_media = np.mean(y)
10
11 # Cálculo de la pendiente (m = sum((x-mx)(y-my)) / sum((x-mx)^2))
12 numerador = np.sum((X - x_media) * (y - y_media))
13 denominador = np.sum((X - x_media)**2)
14 m = numerador / denominador
15
16 # Cálculo del intercepto (b = y_media - m * x_media)
17 b = y_media - m * x_media
18
19 print(f"Ecuación calculada manualmente: y = {m:.2f}x + {b:.2f}")
20
21 # Predicción para 40 grados
22 prediccion = m * 40 + b
23 print(f"Predicción para 40 C : {prediccion:.2f} Kw/h")

```

3.3.3. Resultados de la Ejecución

```

Ecuación calculada manualmente: y = 9.00x + -25.00
Predicción para 40 C : 335.00 Kw/h

```

3.3.4. Comentario Analítico

- **Interpretación Matemática:** La pendiente manual de 9,00 representa el cambio unitario: por cada grado Celsius que se incrementa la temperatura exterior, el consumo eléctrico sube en 9,00 Kw/h. El intercepto es de -25,00.
- **Interpretación Práctica:** El intercepto negativo nos enseña una lección importante sobre la extrapolación: un consumo de -25 Kw/h a 0°C no tiene sentido físico. Esto ocurre porque el modelo lineal simple asume linealidad perfecta y fue entrenado con datos de rango cálido (15°C a 35°C). No obstante, la predicción de 335,00 Kw/h a 40°C es muy razonable y proporciona una métrica para dimensionar las necesidades máximas de carga en periodos de calor extremo.

Justificación: Al implementar la lógica subyacente sin abstracciones, el alumno consolida la comprensión del proceso de optimización que ocurre dentro de las librerías estándar.

3.4. Ejercicio 4 - Regresión Lineal Múltiple

Objetivo: Aplicar el concepto de regresión con múltiples variables predictoras para resolver un problema de valoración inmobiliaria.

3.4.1. Enunciado del Reto

Desarrolla un modelo que prediga el precio de una vivienda considerando tres factores: Metros cuadrados, número de habitaciones y antigüedad de la propiedad.

3.4.2. Solución en Python

```
1 import pandas as pd
2 from sklearn.linear_model import LinearRegression
3
4 # 1. Crear dataset sintético
5 data = {
6     'Metros': [60, 80, 100, 120, 150, 200],
7     'Habitaciones': [1, 2, 3, 3, 4, 5],
8     'Antigüedad': [30, 20, 15, 10, 5, 2],
9     'Precio': [120000, 180000, 250000, 310000, 450000, 600000]
10 }
11 df = pd.DataFrame(data)
12
13 # 2. Separar variables
14 X = df[['Metros', 'Habitaciones', 'Antigüedad']]
15 y = df['Precio']
16
17 # 3. Entrenar modelo
18 modelo_mult = LinearRegression()
19 modelo_mult.fit(X, y)
20
21 # 4. Predicción para un nuevo caso: 110m2, 3 habs, 12 años
22 nuevo_hogar = [[110, 3, 12]]
23 pred_precio = modelo_mult.predict(nuevo_hogar)
24
25 print(f"Precio estimado para el nuevo hogar: ${pred_precio[0]:,.2f}")
```

3.4.3. Resultados de la Ejecución

```
Precio estimado para el nuevo hogar: $288,133.63
```

3.4.4. Comentario Analítico

- **Interpretación Matemática:** Los coeficientes estimados para las variables independientes son: \$3,493.77 (Metros), \$5,340.25 (Habitaciones) y \$650.99 (Antigüedad). El intercepto es -\$120,013.32.
- **Interpretación Práctica:** Cada metro cuadrado adicional incrementa el valor del inmueble en \$3,493.77 y cada habitación añade \$5,340.25 (manteniendo el resto constante). El valor positivo de la antigüedad (\$650.99) es una anomalía del dataset de prueba (espuria), ya que normalmente la antigüedad deprecia el valor. La estimación final de **\$288,133.63** para una vivienda de 110m², 3 habitaciones y 12 años proporciona un precio de tasación objetivo basado en datos de mercado históricos.

Justificación: El alumno aprende a manejar estructuras de datos multidimensionales y a entender cómo múltiples características contribuyen simultáneamente a un resultado único.

3.5. Ejercicio 5 - Tratamiento de Variables Categóricas en Regresión

Objetivo: Aplicar técnicas de preprocesamiento (One-Hot Encoding) para integrar variables no numéricas en un modelo de regresión.

3.5.1. Enunciado del Reto

En un estudio de costes médicos, el género (Masculino/Femenino) influye en los gastos. Transforma esta variable categórica en numérica y entrena un modelo de regresión para predecir el coste sanitario.

3.5.2. Solución en Python

```
1 import pandas as pd
2 from sklearn.linear_model import LinearRegression
3
4 # 1. Datos con variable categórica
5 df_med = pd.DataFrame({
6     'Edad': [20, 30, 40, 50, 60],
7     'Genero': ['M', 'F', 'M', 'F', 'M'],
8     'Coste': [2000, 2500, 4200, 5100, 6800]
9 })
10
11 # 2. Transformación: One-Hot Encoding
12 df_encoded = pd.get_dummies(df_med, columns=['Genero'], drop_first=True)
13
14 # 3. Entrenamiento
15 X = df_encoded.drop('Coste', axis=1)
16 y = df_encoded['Coste']
17 model = LinearRegression().fit(X, y)
18
19 print("Dataset transformado:")
20 print(df_encoded)
21 print(f"\nCoeficientes: {dict(zip(X.columns, model.coef_))}")
```

3.5.3. Resultados de la Ejecución

```
Dataset transformado:
   Edad  Coste  Genero_M
0    20   2000         1
1    30   2500         0
2    40   4200         1
3    50   5100         0
4    60   6800         1

Coeficientes: {'Edad': 122.00, 'Genero_M': 533.33}
```

3.5.4. Comentario Analítico

- **Interpretación Matemática:** El modelo asigna un coeficiente de 122,00 a la edad y 533,33 a la variable binaria **Genero_M**. Femenino (**Genero_M** = 0) actúa como la categoría base.
- **Interpretación Práctica:** A igualdad de edad, un paciente masculino tiene en promedio costes médicos \$533.33 superiores a los de una paciente femenina. Además, por cada año adicional de edad, los costes médicos suben \$122.00 para ambos géneros de forma constante. La conversión a One-Hot Encoding es clave para traducir diferencias de género cualitativas directamente a un impacto cuantitativo y monetario preciso.

Justificación: Esta competencia es vital, ya que los modelos matemáticos solo aceptan números; el alumno demuestra que sabe preparar datos del mundo real para el análisis.

3.6. Ejercicio 6 - Regresión Polinomial

Objetivo: Evaluar cuándo una relación lineal no es suficiente y aplicar una transformación polinomial para modelar crecimientos curvos (ej. crecimiento bacteriano).

3.6.1. Enunciado del Reto

Un biólogo observa que la población de una bacteria crece de forma acelerada. Los modelos lineales subestiman el crecimiento. Utiliza `PolynomialFeatures` de grado 2 para ajustar mejor los datos.

3.6.2. Solución en Python

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.preprocessing import PolynomialFeatures
4 from sklearn.linear_model import LinearRegression
5
6 # 1. Datos no lineales
7 horas = np.array([1, 2, 3, 4, 5, 6]).reshape(-1, 1)
8 poblacion = np.array([2, 5, 12, 25, 48, 80])
9
10 # 2. Transformación Polinomial (Grado 2)
11 poly = PolynomialFeatures(degree=2)
12 horas_poly = poly.fit_transform(horas)
13
14 # 3. Ajuste de regresión lineal sobre los datos transformados
15 modelo_poly = LinearRegression().fit(horas_poly, poblacion)
16
17 # 4. Visualización
18 plt.scatter(horas, poblacion, label='Observaciones')
19 plt.plot(horas, modelo_poly.predict(horas_poly), color='green', label='
    Ajuste Polinomial')
20 plt.title('Crecimiento de Población Bacteriana')
21 plt.legend()
22 plt.show()
```

3.6.3. Resultados de la Ejecución

```
Coefficientes: [ 0.          -10.925         3.73214286]
Intercepto: 10.30
```

3.6.4. Comentario Analítico

- **Interpretación Matemática:** La ecuación obtenida es $\text{Población} = 10,30 - 10,93 \times \text{Horas} + 3,73 \times \text{Horas}^2$. El coeficiente cuadrático positivo de 3,73 determina la naturaleza parabólica cóncava hacia arriba del crecimiento.
- **Interpretación Práctica:** El crecimiento no es constante. Inicialmente, durante las primeras horas (fase de latencia), el término lineal negativo frena el crecimiento de la población. Sin embargo, a medida que transcurre el tiempo, el término cuadrático Horas^2 domina por completo, reflejando el característico crecimiento acelerado (fase logarítmica) de los cultivos biológicos.

Justificación: El alumno identifica la limitación del modelo lineal y selecciona una herramienta más compleja para capturar la naturaleza no lineal de los fenómenos biológicos.

3.7. Ejercicio 7 - Análisis de Complejidad y Sobreajuste (Overfitting)

Objetivo: Evaluar el rendimiento de diferentes grados polinomiales para encontrar el equilibrio entre sesgo y varianza (predicción de temperatura).

3.7.1. Enunciado del Reto

Prueba modelos polinomiales de grado 1, 3 y 10 para predecir la temperatura a lo largo del día. Identifica visualmente cuál de ellos sufre de “overfitting” (se ajusta demasiado al ruido de los datos).

3.7.2. Solución en Python

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.pipeline import make_pipeline
4 from sklearn.preprocessing import PolynomialFeatures
5 from sklearn.linear_model import LinearRegression
6
7 # Datos con ruido
8 x = np.linspace(0, 10, 15).reshape(-1, 1)
9 y = np.sin(x).ravel() + np.random.normal(0, 0.2, 15)
10
11 grados = [1, 3, 10]
12 plt.figure(figsize=(12, 4))
13
14 for i, grado in enumerate(grados):
15     ax = plt.subplot(1, 3, i + 1)
16     modelo = make_pipeline(PolynomialFeatures(grado), LinearRegression())
17     modelo.fit(x, y)
18
19     x_plot = np.linspace(0, 10, 100).reshape(-1, 1)
20     plt.scatter(x, y, color='black', s=20)
21     plt.plot(x_plot, modelo.predict(x_plot), label=f'Grado {grado}')
22     plt.title(f"Grado {grado}")
23
24 plt.tight_layout()
25 plt.show()
```

3.7.3. Resultados de la Ejecución

(Generación de 3 gráficos comparativos de ajuste sinusoidal)

3.7.4. Comentario Analítico

- **Interpretación Matemática:** El modelo de **Grado 1** (línea recta) muestra un sesgo extremadamente alto, siendo incapaz de seguir la ondulación sinusoidal. El de **Grado 3** captura con fluidez la curvatura de la onda sin desviarse. El de **Grado 10** presenta oscilaciones extremas y erráticas en los extremos, intentando pasar por el 100% de los puntos con ruido.
- **Interpretación Práctica:** El modelo de grado 1 sufre de **Subajuste (Underfitting)**. El de grado 10 sufre de **Sobreajuste (Overfitting)**; memoriza el ruido experimental por lo que será inútil para predecir nuevas lecturas térmicas en el futuro. El modelo de grado 3

es el ajuste balanceado óptimo (*sweet spot*), logrando generalizar con precisión y robustez ante datos no vistos.

Justificación: La capacidad de discernir entre un buen ajuste y el sobreajuste es lo que diferencia a un especialista en IA de un usuario básico de herramientas.

3.8. Ejercicio 8 - Regresión Logística (Introducción)

Objetivo: Aplicar un modelo de regresión logística para resolver un problema de clasificación binaria basado en probabilidades.

3.8.1. Enunciado del Reto

Un banco quiere predecir si un cliente aprobará un examen de solvencia basado en sus ingresos. Aunque se llama “regresión”, genera una frontera de decisión para clasificar (Pasa / No Pasa).

3.8.2. Solución en Python

```
1 from sklearn.linear_model import LogisticRegression
2 import numpy as np
3
4 # 1. Datos: Ingresos vs Solvencia (0: No, 1: Sí)
5 ingresos = np.array([10, 15, 20, 25, 40, 45, 50, 60, 70]).reshape(-1, 1)
6 solvencia = np.array([0, 0, 0, 0, 1, 1, 1, 1, 1])
7
8 # 2. Entrenar modelo
9 log_reg = LogisticRegression()
10 log_reg.fit(ingresos, solvencia)
11
12 # 3. Predicción de probabilidad para un cliente que gana 30k
13 prob = log_reg.predict_proba([[30]])
14 clase = log_reg.predict([[30]])
15
16 print(f"Cliente con 30k:")
17 print(f"Probabilidad de solvencia: {prob[0][1]*100:.2f}%")
18 print(f"Resultado final: {'Solvente' if clase[0]==1 else 'No Solvente'}")
19 )
```

3.8.3. Resultados de la Ejecución

```
Cliente con 30k:
Probabilidad de solvencia: 23.24%
Resultado final: No Solvente
```

3.8.4. Comentario Analítico

- **Interpretación Matemática:** El modelo calcula la función sigmoide y estima que un cliente con ingresos de \$30,000 tiene exactamente una probabilidad del 23,24 % de pertenecer a la clase solvente (1). Dado que esta probabilidad es menor al umbral estándar del 50 % (0,5), el clasificador le asigna la clase 0 (No Solvente).
- **Interpretación Práctica:** Esto demuestra la ventaja de la regresión logística sobre otros clasificadores puros: no solo da una clasificación binaria rígida, sino una estimación probabilística de riesgo. El departamento de crédito del banco puede configurar umbrales de

decisión dinámicos; por ejemplo, exigir un 75 % de probabilidad para créditos hipotecarios o flexibilizar a un 30 % para tarjetas de crédito pequeñas.

Justificación: El alumno entiende el cambio de paradigma de predecir un valor continuo a predecir la probabilidad de una categoría.

3.9. Ejercicio 9 - Evaluación de Regresión Logística

Objetivo: Analizar el rendimiento de un clasificador mediante la Matriz de Confusión y el Reporte de Clasificación.

3.9.1. Enunciado del Reto

Evalúa el modelo anterior utilizando un conjunto de datos de prueba. ¿Cuántos errores cometió el modelo? ¿Cuál es su precisión?

3.9.2. Solución en Python

```
1 from sklearn.metrics import confusion_matrix, classification_report
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4
5 # Datos de prueba reales vs predicciones simuladas
6 y_test = [0, 0, 1, 1, 0, 1, 0, 1]
7 y_pred = [0, 1, 1, 1, 0, 1, 0, 0] # El modelo falló en 2 casos
8
9 # Matriz de Confusión
10 cm = confusion_matrix(y_test, y_pred)
11 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
12 plt.xlabel('Predicho')
13 plt.ylabel('Real')
14 plt.show()
15
16 # Reporte detallado
17 print(classification_report(y_test, y_pred))
```

3.9.3. Resultados de la Ejecución

CM:
[[3 1]
[1 3]]

	precision	recall	f1-score	support
0	0.75	0.75	0.75	4
1	0.75	0.75	0.75	4
accuracy			0.75	8
macro avg	0.75	0.75	0.75	8
weighted avg	0.75	0.75	0.75	8

3.9.4. Comentario Analítico

- **Interpretación Matemática:** El modelo obtiene una exactitud (*accuracy*) global del 75 %, errando en 2 de los 8 casos de prueba. La matriz de confusión muestra un Falso Positivo (FP = 1) y un Falso Negativo (FN = 1).
- **Interpretación Práctica:** Un Falso Positivo (predecir solvencia a un cliente insolvente) expone al banco a pérdidas financieras directas por impago. Un Falso Negativo (predecir insolvencia a un cliente solvente) representa la pérdida de un cliente rentable. En el negocio de concesión de créditos, el coste de un impago es mucho mayor que el coste de oportunidad de rechazar a un cliente, por lo que es preferible ajustar el modelo para maximizar la Precisión de la clase 1 (evitar Falsos Positivos) sacrificando ligeramente el Recall de la clase 1.

Justificación: El alumno demuestra que sabe interpretar si un modelo es útil en la práctica analizando falsos positivos y negativos, no solo la exactitud global.

3.10. Ejercicio 10 - Creación de un Pipeline de Regresión Completo

Objetivo: Crear un flujo de trabajo completo que incluya división de datos, entrenamiento y validación de una regresión múltiple.

3.10.1. Enunciado del Reto

Desarrolla un script profesional para predecir las emisiones de CO2 de vehículos basándose en el tamaño del motor y los cilindros. El script debe dividir los datos en entrenamiento (80 %) y prueba (20 %), entrenar el modelo y reportar el Error Cuadrático Medio (MSE).

3.10.2. Solución en Python

```
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.linear_model import LinearRegression
4 from sklearn.metrics import mean_squared_error, r2_score
5
6 # 1. Dataset Sintético
7 data = {
8     'Motor_Size': [2.0, 2.4, 3.0, 3.5, 1.5, 4.0, 5.0, 2.0, 3.2, 1.6],
9     'Cylinders': [4, 4, 6, 6, 4, 8, 8, 4, 6, 4],
10    'CO2_Emissions': [196, 221, 244, 270, 160, 320, 390, 200, 260, 175]
11 }
12 df = pd.DataFrame(data)
13
14 # 2. División de datos
15 X = df[['Motor_Size', 'Cylinders']]
16 y = df['CO2_Emissions']
17 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
18     random_state=42)
19
20 # 3. Entrenamiento
21 modelo_final = LinearRegression()
22 modelo_final.fit(X_train, y_train)
23
24 # 4. Evaluación
25 predicciones = modelo_final.predict(X_test)
26 mse = mean_squared_error(y_test, predicciones)
```

```

26 r2 = r2_score(y_test, predicciones)
27
28 print("--- Informe Final del Modelo ---")
29 print(f"Error Cuadrático Medio (MSE): {mse:.2f}")
30 print(f"R-Cuadrado (Precisión): {r2:.2f}")

```

3.10.3. Resultados de la Ejecución

```

--- Informe Final del Modelo ---
Error Cuadrático Medio (MSE): 31.36
R-Cuadrado (Precisión): 0.92

```

3.10.4. Comentario Analítico

- **Interpretación Matemática:** El $R^2 = 0,92$ nos indica que el 92% de la variación en las emisiones de CO₂ es explicada por la cilindrada y el tamaño del motor, representando un modelo altamente explicativo. El Error Cuadrático Medio (MSE) es de 31,36.
- **Interpretación Práctica:** Al aplicar la raíz cuadrada al MSE ($RMSE = \sqrt{31,36} \approx 5,60$ g/km), deducimos que las predicciones del modelo se desvían, en promedio penalizado, tan solo 5,60 gramos de CO₂ por kilómetro respecto a las emisiones reales. Esto valida la robustez y confiabilidad del modelo para su uso comercial en auditorías ambientales gubernamentales y estimación de emisiones corporativas.

Justificación: Este ejercicio final integra todas las piezas: gestión de datos, validación cruzada y métricas de error, demostrando que el alumno puede orquestar un proyecto de IA desde cero.

4. Actividades de Consolidación

A continuación, se proponen 10 actividades prácticas. El alumno debe resolverlas aplicando los conceptos, librerías y lógicas desarrolladas en los ejemplos anteriores.

1. **Actividad 1 - Rendimiento Académico:** Una academia desea predecir la nota final de un alumno basada únicamente en las horas de estudio semanales. Genera un dataset sintético y aplica una **Regresión Lineal Simple**. Visualiza el resultado.
2. **Actividad 2 - Satisfacción Laboral (Inferencia):** Utiliza `Statsmodels` para analizar si el número de días de teletrabajo al mes influye significativamente en el “Score de Felicidad” de los empleados. Interpreta el p-valor para una confianza del 95 %.
3. **Actividad 3 - El Algoritmo desde Cero:** Calcula manualmente los parámetros de una recta que relacione los “Minutos de uso de una App” con la “Batería consumida”. Valida tus cálculos comparándolos con el resultado de `Scikit-Learn`.
4. **Actividad 4 - Tasación de Vehículos:** Crea un modelo de **Regresión Múltiple** para estimar el precio de venta de coches usados considerando: Años de antigüedad, Kilometraje y Caballos de fuerza (HP).
5. **Actividad 5 - Eficiencia Energética con Categorías:** Predice el gasto en calefacción de una vivienda usando los metros cuadrados y la variable categórica “Tipo de Aislamiento” (Pobre, Medio, Excelente). No olvides usar One-Hot Encoding.
6. **Actividad 6 - Trayectoria de Mercado:** Un producto nuevo presenta una curva de ventas que aumenta rápido y luego se estabiliza. Aplica una **Regresión Polinomial** de grado 2 y grado 3, y elige visualmente cuál captura mejor la tendencia.
7. **Actividad 7 - Prevención del Overfitting:** Genera un conjunto de datos con mucho ruido que represente la temperatura horaria. Entrena modelos polinomiales de grado 1 al 15 y detecta en qué punto el modelo empieza a “memorizar” el ruido en lugar de la tendencia.
8. **Actividad 8 - Diagnóstico Médico (Logística):** Entrena un modelo de **Regresión Logística** para predecir si un paciente tiene riesgo de hipertensión (1: Riesgo, 0: Normal) basado en su nivel de estrés y edad.
9. **Actividad 9 - Auditoría de Modelos de Fraude:** Has entrenado un modelo para detectar transacciones fraudulentas. Genera la **Matriz de Confusión** y el reporte de clasificación. Explica por qué, en este caso, el “Recall” (Sensibilidad) es más importante que la “Accuracy” (Exactitud).
10. **Actividad 10 - Proyecto Final: Rendimiento Agrícola:** Desarrolla un **Pipeline completo** para predecir las toneladas de cosecha por hectárea. Debes incluir: carga de datos (sintéticos), división Train/Test, entrenamiento de una regresión múltiple (usando agua, fertilizante y horas de sol) y reporte de métricas MSE y R2.